

SOFIAA OS

Análisis Doctoral

O R Á C U L O

Predictive Intelligence Engine

El oráculo que anticipa antes de que el Olimpo pregunte

Versión 1.0 — Sprints O-1 · O-2 · O-3 · O-4 · O-5
Julio 2026

1. Resumen Ejecutivo

ORACULO es el motor de inteligencia predictiva de SOFIAA OS — el septimo dios del Olimpo y el primero de Generacion 2. Su nombre refleja con exactitud su funcion: no genera ni ejecuta, sino que anticipa. Sintetiza senales de todos los engines operacionales del sistema (ATENA, TEC Bii, PROMETEO, NEXO, HERMES, THEMIS) y las transforma, mediante un motor de reglas determinista, en predicciones accionables persistidas en Firestore.

La premisa de diseno de ORACULO parte de una observacion critica sobre los sistemas de IA operacionales: la mayoría de las herramientas responden bien cuando se les pregunta, pero ninguna anticipa antes de que el problema escale. ORACULO invierte esa ecuacion — escanea el estado real del sistema cada 6 horas, identifica patrones de riesgo cruzados entre engines, y los presenta al usuario antes de que pregunte.

ORACULO no genera — anticipa. Motor determinista, sin LLM en el hot path. Sintetiza senales de todos los dioses del Olimpo.

El sistema fue implementado en cinco sprints consecutivos (O-1 a O-5) que construyeron, respectivamente: los scanners de ATENA y TEC Bii, los scanners de PROMETEO, NEXO, HERMES y THEMIS mas el motor de pronosticos, la interfaz de usuario completa, los insights estrategicos cross-engine con Groq, y la inyeccion de contexto en el chat mas la automatizacion CRON.

2. Contexto y Motivacion

Cuando SOFIAA OS alcanzo Generacion 2 — con seis extensions activas, HERMES ejecutando acciones autonomas, y THEMIS auditando cada decision — emergio un desafio nuevo: la cantidad de datos operacionales que el sistema producía superaba la capacidad de cualquier usuario de monitorearlos manualmente. Un director de TEC Bii no puede revisar cada proyecto, cada proveedor, cada violacion SPC, cada objetivo PROMETEO y cada veredicto THEMIS en tiempo real.

ORACULO nacio como la respuesta sistémica a ese desafio. En lugar de agregar mas dashboards, la arquitectura propone un meta-observador: un engine que lee el estado de todos los demas y produce un unico flujo de predicciones priorizadas por severidad y horizonte temporal. El usuario no necesita revisar seis sistemas — ORACULO ya lo hizo.

La decision de mantener el motor de predicciones completamente determinista (sin LLM) y reservar Groq unicamente para la sintesis de insights estrategicos cross-engine (Sprint O-4) refleja una disciplina arquitectonica: la prediccion debe ser reproducible, auditable y rapida. Los insights estrategicos, en cambio, requieren razonamiento narrativo sobre patrones complejos — ahi el LLM agrega valor real.

3. Arquitectura del Sistema

3.1 Flujo Principal

El ciclo completo de ORACULO opera en tres etapas secuenciales:

- Scan: `scanAllEngines()` consulta los 6 engines en paralelo (`Promise.all`) y recopila `OracleSignal[]` crudas. Target < 500ms.

- Predict: generatePredictions() agrupa senales por engine + signalType + entityId, deduplica contra predicciones activas existentes, aplica el motor de reglas, pasa por THEMIS gate, y persiste predicciones aprobadas en Firestore.
- Insight: generateInsights() (Sprint O-4) envia las predicciones activas a Groq para sintesis narrativa cross-engine. Es el unico paso que usa LLM.

3.2 Configuracion Global

ORACULO es configurable via Firestore (system/oraculo) con cache de 2 minutos. La OracleConfig define que engines estan activos (enginesEnabled), los umbrales numericos por engine (thresholds), y el intervalo de escaneo automatico (scanIntervalHours, default 6h). Si Firestore no responde, DEFAULT_ORACLE_CONFIG se aplica silenciosamente.

3.3 Tabla de Senales por Engine

Engine	Signal Type	Categoría	Sev.	Umbral de disparo
ATENA	amef_npm_alto	operational_risk	warn/critical	NPR >= 100 (critico >= 200), estado abierto
ATENA	spc_violation	operational_risk	warn/critical	Violaciones WE > 0 (critico >= 3)
TEC Bii	high_urgency_project	delivery_risk	warn/critical	urgencyScore >= 0.75 (critico >= 0.90)
TEC Bii	proveedor_riesgo	resource_risk	warning	alertaRiesgo === true en Firestore
PROMETEO	goal_at_risk	marketing_risk	warn/critical	Desviacion >= 20% del objetivo (critico >= 40%)
PROMETEO	creative_fatigue	marketing_risk	warning	>= 3 creativos con performance < 30/100
NEXO	hypothesis_pending	strategic_opportunity	warn/critical	Confidence >= 0.70, no validada/descartada
HERMES	hermes_veto_ratio	compliance_risk	warn/critical	Ratio vetos >= 10% en 30 dias (critico >= 25%)
THEMIS	themis_violations	compliance_risk	warn/critical	> 3 veredictos error en 48h (critico >= 8)

3.4 Motor de Predicciones — Reglas Deterministas

Una vez recopiladas las senales, generatePredictions() opera en cinco pasos por grupo:

Deduplicacion: isDuplicate() verifica que no exista prediccion activa del mismo engine + signalType + entityId en Firestore. Previene spam de predicciones redundantes.

buildTitle(): Genera el titulo en lenguaje natural desde el tipo de senal y los datos del payload. Completamente determinista — sin LLM.

buildSummary(): Genera el resumen de 2-3 oraciones con datos concretos del payload: NPR exacto, % de desviacion, dias restantes, etc.

buildRecommendations(): Genera 1-3 recomendaciones tipadas (hermes_action | user_decision | review) con payloads listos para enviar a HERMES si aplica.

THEMIS gate: applyThemisGate() evalua el texto de la prediccion contra las politicas cognitivas activas. Una prediccion no aprobada no se persiste.

4. Scanners por Engine

4.1 ATENA Scanner

Consulta dos colecciones de Firestore en paralelo. Para AMEF: busca items con estado abierto o en_proceso y NPR >= 100 (umbral configurable). Items con NPR >= 200 generan senales critical; entre 100-

199, warning. Para SPC: escanea todas las cartas de control y detecta variables con violaciones Western Electric > 0. Tres o mas violaciones en la misma variable constituyen senal critical.

4.2 TEC Bii Scanner

Escanea proyectos con urgencyScore >= 0.75 (critical si >= 0.90), capturando deadline en dias si esta disponible. Tambien detecta proveedores con alertaRiesgo === true en Firestore, generando senales de resource_risk con calidad promedio, tasa de cumplimiento y tendencia.

4.3 PROMETEO Scanner

Opera sobre los workspaces de smm_workspaces para acceder a datos multicliente. Detecta dos patrones: BrandGoals activos con desviacion >= 20% del valor objetivo (critical si >= 40%), y fatiga creativa cuando tres o mas creativos de Creative Memory tienen performanceScore < 30 y usarDeNuevo === false.

4.4 NEXO Scanner

Recorre las colecciones de entidades de TEC Bii (proyectos, proveedores, empleados) y extrae las hipotesis cross-domain generadas por el motor de razonamiento cruzado. Filtra hipotesis con confidence >= 0.70 que no han sido validadas ni descartadas. La hipotesis de mayor confianza se incluye verbatim en el payload.

4.5 HERMES Scanner

Analiza la cola de acciones de los ultimos 30 dias en cada workspace. Calcula el ratio de acciones con estado vetada_por_themis sobre el total. Un ratio >= 10% genera senal warning; >= 25% genera critical. Esta senal de compliance_risk indica que las intenciones autonomas de HERMES estan chocando sistematicamente con las politicas de THEMIS.

4.6 THEMIS Scanner

Consulta los veredictos de THEMIS con severity === error en las ultimas 48 horas usando el campo date (YYYY-MM-DD) para eficiencia de query. Si hay mas de 3 veredictos de error, agrega las politicas unicas afectadas y genera una senal de compliance_risk. El umbral critico es 8 o mas veredictos en la ventana de 48 horas.

5. Tipos y Contratos

5.1 OracleSignal

La senal cruda es el atomo de informacion de ORACULO. Contiene: sourceEngine, signalType, severity (info | warning | critical), entityId y entityLabel para mostrar al usuario, category (la categoria de riesgo inferida), payload con los datos crudos del engine, y capturedAt timestamp.

5.2 OraclePrediction

La prediccion es la unidad de valor de ORACULO. Contiene titulo y summary en lenguaje natural, confidence calculado por reglas (0.0-1.0), horizon (24h | 7d | 30d | 90d), recommendations tipadas con payloads de accion, status (active | acknowledged | resolved | dismissed), y los campos themisApproved + themisVerdictId para trazabilidad.

5.3 OracleForecast

Los pronosticos de series de tiempo se calculan por regresion lineal sobre puntos historicos. Los tres pronosticos implementados son: spc_violations (ATENA, horizonte 7 dias), urgency_score_avg (TEC Bii, horizonte 30 dias), y goal_deviation_avg (PROMETEO, horizonte 30 dias). La confianza aumenta con mas puntos de datos: minimo 2 puntos -> 50%, maximo configurado en 90%.

5.4 OracleInsight

Los insights son narrativas estrategicas cross-engine generadas por Groq. Cada insight tiene titulo (max 80 chars), body (2-4 oraciones de analisis estrategico), relatedPredictionIds (solo IDs validados contra el set actual — previene alucinaciones), y confidence.

6. Motor de Confianza y Horizonte

6.1 computeConfidence()

La confianza de una prediccion se calcula por la combinacion de senales en el grupo:

>= 2 senales critical: 0.95

1 critical + >= 1 warning: 0.85

1 critical sola: 0.75

>= 3 warnings: 0.80

>= 2 warnings: 0.70

1 warning sola: 0.55

6.2 computeHorizon()

El horizonte se determina por el tipo de senal y los valores del payload:

- high_urgency_project: 24h si urgencyScore >= 0.90, 7d si menor.
- amef_npm_alto: 7d si NPR >= 200 (critico), 30d si menor.
- spc_violation: 24h si >= 5 violaciones, 7d si menos.
- goal_at_risk: 7d si desviacion >= 40%, 30d si menor.
- hypothesis_pending: 7d si confidence >= 0.90, 30d si menor.
- hermes_veto_ratio, creative_fatigue, proveedor_riesgo: siempre 30d.
- themis_violations: siempre 7d.

Cuando un grupo tiene multiples senales con horizontes distintos, groupHorizon() toma el mas urgente (menor indice en el orden 24h -> 7d -> 30d -> 90d).

7. Insights Estrategicos — Sprint O-4

El unico punto donde ORACULO usa un LLM (Groq llama-3.3-70b-versatile) es en la generacion de insights estrategicos cross-engine. La justificacion es arquitectonica: identificar patrones sistemicos entre predicciones de engines distintos requiere razonamiento narrativo que un motor de reglas no puede capturar de forma generalizable.

El flujo de generateInsights() opera en cinco pasos: (1) Fetch de hasta 30 predicciones activas ordenadas por timestamp descendente. (2) Construccion de un resumen compacto por prediccion (max 200 chars de summary) para controlar el uso de tokens. (3) Llamada a Groq en JSON mode con instruccion de generar 1-3 insights que crucen al menos 2 engines distintos. (4) Validacion de IDs: los relatedPredictionIds retornados por Groq se filtran contra el set real de predicciones activas. (5) Batch write a Firestore.

La sintesis por LLM actua sobre predicciones ya validadas y persistidas — nunca sobre datos crudos. Groq razona sobre hechos, no sobre senales.

8. Integracion con HERMES y el Chat

8.1 Recomendaciones con Payload HERMES

Las recomendaciones de tipo hermes_action incluyen un hermesPayload estructurado listo para enviarse a POST /api/hermes/execute. Esto crea un circuito completo: ORACULO detecta un AMEF critico -> genera recomendacion con payload de crear tarea en Monday.com -> el usuario aprueba -> HERMES ejecuta -> THEMIS valida. La prediccion se convierte en accion sin que el usuario escriba codigo.

8.2 Inyeccion en el Chat — Sprint O-5

ORACULO inyecta un bloque de contexto en el system prompt de SOFIAA mediante buildOraculoBlock() en chat/route.ts. El bloque incluye las ultimas 3 predicciones activas mas criticas con sus titulos, severidades, horizontes y resúmenes. SOFIAA puede responder preguntas sobre el estado del sistema con informacion actualizada sin que el usuario navegue a /oraculo.

8.3 CRON Automatico — Sprint O-5

vercel.json registra dos CRON jobs: GET /api/cron/oraculo/scan cada 6 horas (escaneo de senales y generacion de predicciones) y GET /api/cron/oraculo/insights cada 24 horas (sintesis narrativa con Groq). El ciclo automatico garantiza que ORACULO siempre tenga predicciones frescas sin intervencion manual.

9. Interfaz de Usuario

La UI de ORACULO sigue el theme purpura (#6D28D9) y esta accesible en /oraculo con un sidebar de navegacion. Se compone de cuatro paginas:

Centro de Mando (/oraculo): KPIs en tiempo real (predicciones activas, criticas, warnings, resueltas), boton de scan manual, y ultima actividad del sistema.

Predicciones (/oraculo/predicciones): Lista filtrable por severidad y categoria. Cada prediccion muestra confidence, horizon, senales fuente, y recomendaciones. Acciones: acknowledge, dismiss, resolve con nota opcional.

Pronosticos (/oraculo/forecasts): Panel agrupado por engine con tarjetas de valor actual vs proyectado, tendencia (subida/bajada/estable), y barra de confianza del pronostico.

Insights (/oraculo/insights): Tarjetas expandibles con narrativa estrategica generada por Groq. Muestra predicciones relacionadas y barra de confianza.

10. Metricas Operacionales

Latencia de scanAllEngines(): < 500ms (6 engines en paralelo, Promise.all)

Latencia de generatePredictions(): < 200ms por prediccion (reglas deterministas + THEMIS gate)

Latencia de generateInsights(): 2-5 segundos (incluye llamada Groq)

TTL de cache de configuracion: 120 segundos (system/oraculo)

Frecuencia de scan CRON: Cada 6 horas

Frecuencia de insights CRON: Cada 24 horas

Engines monitoreados: 6 (ATENA, TEC Bii, PROMETEO, NEXO, HERMES, THEMIS)

Tipos de senal implementados: 9 signal types, 6 categorias de riesgo

Colecciones Firestore: oracle_predictions, oracle_signals, oracle_forecasts, oracle_insights

11. Estado Actual y Hoja de Ruta

11.1 Implementado (v1.0)

- O-1: Scanners ATENA (AMEF + SPC) y TEC Bii (proyectos + proveedores). Motor de reglas base. THEMIS gate.
- O-2: Scanners PROMETEO, NEXO, HERMES, THEMIS. Motor de pronosticos con regresion lineal (3 metricas).
- O-3: Layout + sidebar /oraculo. Centro de Mando, Predicciones con ciclo de vida completo, Forecasts agrupados por engine.
- O-4: generateInsights() con Groq JSON mode. Validacion anti-alucinacion de IDs. Pagina /oraculo/insights.
- O-5: buildOraculoBlock() en chat/route.ts. CRON automatico en vercel.json (scan 6h + insights 24h).

11.2 Hoja de Ruta Futura

- O-6: Panel de configuracion de umbrales desde la UI (editar thresholds en system/oraculo sin tocar codigo).
- O-7: Ejecucion automatica de recomendaciones por HERMES para predicciones de alta confianza.
- O-8: Motor de aprendizaje — registrar que predicciones fueron utiles vs descartadas para ajustar umbrales.
- O-9: Anomaly detection — detectar cambios bruscos en metricas mas alla de la regresion lineal.
- O-10: Integracion APOLO — las predicciones resueltas se incluyen en los reportes ejecutivos semanales.

12. Conclusion Doctoral

ORACULO representa la madurez arquitectonica de SOFIAA OS. Los seis engines anteriores son motores de capacidad — cada uno resuelve un dominio especifico con excelencia. ORACULO es el primer engine cuyo valor no esta en lo que hace, sino en lo que observa. Es el meta-nivel del sistema: la capa que transforma datos operacionales distribuidos en inteligencia predictiva unificada.

La decision de diseno mas importante de ORACULO es la separacion de niveles de razonamiento: el escaneo y la prediccion son deterministas, auditables, y reproducibles — el LLM no interviene en la cadena de confianza critica. Groq entra unicamente en la sintesis narrativa cross-engine, donde el razonamiento sobre patrones complejos agrega valor real sin comprometer la integridad de la prediccion subyacente.

La integracion bidireccional con HERMES (recomendaciones con payloads ejecutables) y con THEMIS (gate de predicciones) cierra el ciclo del Olimpo: ORACULO anticipa, HERMES ejecuta, THEMIS audita. El sistema no solo responde — piensa hacia adelante.

ORACULO convierte a SOFIAA OS en un sistema que sabe lo que va a pasar antes de que el usuario lo pregunte. Eso es el salto de Generacion 1 a Generacion 2.

— Equipo SOFIAA OS · Julio 2026